

Persistencia del Estado del Mundo y Sistemas de LOD de Actores en Unreal Engine

Este documento de información resume los puntos clave, metodologías y avances técnicos presentados en relación con la persistencia del estado del mundo y los sistemas de Nivel de Detalle (LOD) de actores, con un enfoque particular en las capacidades introducidas en Unreal Engine (UE) 5.8.

Resumen Ejecutivo

La creación de mundos masivos y dinámicos exige un equilibrio crítico entre la fidelidad visual y el rendimiento del sistema. Este equilibrio se logra a través de dos procesos interconectados: la **Persistencia** (el guardado y restauración del estado del mundo) y la **Hidratación** (el intercambio dinámico de representaciones de objetos entre actores, actores de instancia y entidades de Mass). Los avances recientes en Unreal Engine, específicamente el plugin *Level Streaming Persistence* (LSP) y el sistema de *Instance Actors*, proporcionan un marco de trabajo para manejar miles de objetos interactivos con una carga mínima en el procesador y la memoria. La integración de estos sistemas permite que acciones del jugador, como abrir una puerta, destruir un objeto o aplicar efectos de estado a un NPC, se mantengan de manera coherente incluso después de que los niveles se descarguen de la memoria o el juego se reinicie.

1. Conceptos Fundamentales: Persistencia e Hidratación

La arquitectura de un juego moderno debe ir más allá de simplemente usar actores tradicionales para todos los objetos interactivos.

- **Persistencia:** Se refiere a la capacidad de capturar el estado de un objeto para restaurarlo en una sesión futura o cuando un nivel se vuelve a cargar mediante *level streaming*.
- **Hidratación/Deshidratación:** Es el proceso de cambiar la representación de un objeto basado en su relevancia o distancia.
- Un objeto **hidratado** es un *Actor* completo con lógica de gameplay y componentes detallados.
- Un objeto **deshidratado** puede ser un *Instance Actor* o una entidad de *Mass*, representados visualmente por *Instance Static Meshes* (ISM) para optimizar el rendimiento.

Matriz de Selección de Implementación

Tipo de Objeto, Sistema Recomendado, Caso de Uso Principal

Interactuables generales, Actores Estándar, Objetos únicos con lógica compleja.

Interactuables estacionarios a gran escala, Instance Actors, "Puntos de recolección, objetos destructibles (ej. Lego Fortnite)."

Objetos móviles/spawneables a gran escala, Mass Entities, "Multitudes de NPCs, proyectiles, escombros masivos."

2. Arquitectura de Guardado y Serialización

El sistema se basa en lo que se denomina la "Save Game API", que permite crear clases con propiedades personalizadas que se guardan en archivos de datos.

Identificación de Objetos

Para restaurar el estado, es crucial identificar los objetos de forma única entre sesiones:

- **Path Name:** Útil y estable para actores colocados en el mapa, pero puede cambiar si el actor se mueve a otra capa de *World Partition*.
- **Actor GUID:** Una propiedad de editor que algunos estudios exponen a juegos empaquetados para mayor estabilidad.
- **Gameplay Tags/Nombres personalizados:** Ofrecen el mayor control pero requieren herramientas adicionales para evitar errores manuales.

Serialización de Datos

Existen dos enfoques principales para capturar el estado de un objeto:

1. **Especificador SaveGame :** Se marca una propiedad en C++ o Blueprint. Un archivo de archivo (*archive*) serializa automáticamente estas propiedades en un payload binario. Es robusto y tiene compatibilidad básica hacia atrás, pero tiene un ligero sobrecoste de metadatos.
2. **Funciones de Serialización Personalizadas:** Permiten escribir solo los datos necesarios (ej. solo salud y etiquetas), optimizando el tamaño del archivo, aunque el desarrollador debe gestionar manualmente la compatibilidad de versiones.

3. Level Streaming Persistence (LSP)

Introducido experimentalmente en UE 5.3 y actualizado en UE 5.8, el plugin LSP automatiza la gestión de registros para actores que se cargan y descargan mediante *streaming*.

Características Principales

- **Registro Automático:** Detecta actores y sub-objetos de clases configuradas y crea registros de sus propiedades cuando el nivel se descarga.
- **Restauración Transparente:** Al volver a cargar el nivel, el sistema reaplica los valores antes de que se ejecute el evento BeginPlay.
- **Configuración Opt-in:** Casi toda la funcionalidad es opcional para evitar procesamiento innecesarios en actores que no requieren persistencia.

Hooks de Lógica Personalizada

El sistema permite insertar lógica en momentos específicos del ciclo de vida:

- **Pre-persist:** Antes de capturar los datos.
- **Post-restore:** Después de que los datos se han reaplicado. Es crítico para componentes añadidos mediante Blueprints (SCS/UCS), ya que estos se inician tarde y podrían perder la restauración inicial.

4. Instance Actors: Rendimiento y Escalabilidad

Este sistema (experimental desde UE 5.4) es fundamental para juegos que manejan miles de objetos estacionarios conocidos en tiempo de cocción (*cook time*).

- **Eficiencia:** En pruebas de rendimiento, los *Instance Actors* cargan hasta **5 veces más rápido** que los actores tradicionales y tienen un impacto significativamente menor en los hilos de juego y física.
- **Funcionamiento:** Los objetos lejanos se renderizan como componentes ISM. Al acercarse el jugador, se "hidratan" en actores reales.

- **Persistencia de Instancias:** Los datos se gestionan a través de un *Instance Actor Manager* . Para la persistencia, se recomienda serializar los datos en el objeto de datos de la instancia antes de que el sistema de *Mass* procese las solicitudes de aparición (*spawn*).

5. Aplicaciones Prácticas en Sistemas de Gameplay

Gameplay Ability System (GAS) y State Trees

- **Atributos de GAS:** Se pueden persistir fácilmente marcando las estructuras de atributos para que LSP las rastree.
- **State Trees:** Para restaurar el estado de un NPC, se puede usar la función Force Transition para saltar directamente a un estado específico, evitando la re-evaluación de condiciones de entrada que podrían haber cambiado.

Física y Destrucción (Chaos)

Incluso los sistemas complejos como las colecciones de geometría (*Geometry Collections*) pueden persistirse. Al guardar la ubicación de las partículas de Chaos y qué partes se han roto, es posible reconstruir una escena de escombros exactamente como quedó en la sesión anterior.

Referencias entre Actores

Un desafío común es restaurar referencias entre actores que se han vuelto a generar (*respawned*). La solución propuesta implica:

1. Almacenar el *Path Name* original del actor referido.
2. Usar un subsistema que registre a los actores generados con sus nombres antiguos.
3. Utilizar un *callback* diferido que espere a que el nivel del actor referido termine de cargarse antes de resolver la referencia.

6. Consideraciones Técnicas de UE 5.8

La versión 5.8 introduce mejoras significativas en la seguridad de hilos (*thread safety*) para el sistema de *Mass* . Anteriormente, leer datos de fragmentos de entidades podía ser peligroso si un procesador de *Mass* estaba en ejecución. En 5.8, los procesadores de *Mass* utilizan el hilo de juego de manera más integrada, facilitando la lectura de datos para serialización al final del frame. **Cita Destacada:** "Para crear juegos con alto rendimiento, es importante considerar alternativas a simplemente usar actores; se deben considerar actores de instancia para interactuables estacionarios y entidades de Mass para interactuables móviles a gran escala."